

# From Notebook to Public API — Step by Step

**Today's model:** the AdaBoost classifier from `train_and_serialize_adaboost.py` in `../Predictive-Maintenance-Classification/` — same cleaning, same features, same train/test split as every other model we've built on this dataset. It predicts **Failure vs. No Failure** from 5 sensor readings + machine type.

**Today's real goal isn't the model — it's everything that happens *after* a model is trained.** A notebook that prints "macro-F1 = 0.73" is not something anyone outside this room can use. This guide turns that trained model into something a browser, a phone, or another program can actually call.

```
Browser (public) → Nginx (port 80) (reverse proxy) → Gunicorn (port 8000) (WSGI server) → Flask app (our code) → model.joblib (the trained mo
```

We build this from the inside out: serialize the model → wrap it in Flask → serve an HTML page that calls it → run it for real dev testing → then harden it for production with Gunicorn and Nginx.

## Step 1 — Serializing the model

"Serializing" just means: take a Python object that's sitting in memory (a trained model, a fitted scaler) and write it to disk as a file, so a *different* Python process — like our API, minutes or months later — can load it back and use it immediately, without retraining.

We use `joblib`, not the plain `pickle` module — functionally similar, but `joblib` is noticeably faster for objects that hold big NumPy arrays (which almost every sklearn model does internally).

From `train_and_serialize_adaboost.py`:

```
import joblib

joblib.dump(scaler, MODELS_DIR / "scaler.joblib")
joblib.dump(best_ada_binary_model, MODELS_DIR / "binary_model.joblib")
```

Two files come out of that:

| File                             | What it holds                                                                                                                                                                                                                   |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>scaler.joblib</code>       | The <b>fitted</b> <code>StandardScaler</code> — the exact mean/std it learned from the training data. New requests must be scaled with <i>this</i> , not a fresh scaler, or the numbers won't mean the same thing to the model. |
| <code>binary_model.joblib</code> | The trained <code>AdaBoostClassifier</code> itself — every weak learner and every weight it assigned them.                                                                                                                      |

Both files are already copied into `model/` in this folder, so the API below is self-contained — you don't need to re-run training to follow along.

**Loading them back is one line each:**

```
model = joblib.load("model/binary_model.joblib")
scaler = joblib.load("model/scaler.joblib")
```

That's the entire idea. Everything from here is just: *put that loading code somewhere a network request can reach it.*

---

## Step 2 — Building the Flask API

---

Flask is a **micro** web framework — it does almost nothing by default beyond "map a URL to a Python function." That's exactly what we want here.

Open `app.py`. The important part loads the model **once**, when the server starts — not once per request:

```
model = joblib.load(HERE / "model" / "binary_model.joblib")
scaler = joblib.load(HERE / "model" / "scaler.joblib")
```

Then the actual API route:

```
@app.route("/predict", methods=["POST"])
def predict():
    payload = request.get_json()
    ...
    row_scaled = build_feature_row(payload)
    prediction = int(model.predict(row_scaled)[0])
    failure_probability = float(model.predict_proba(row_scaled)[0][1])
    return jsonify({
        "prediction": "Failure" if prediction == 1 else "No Failure",
        "failure_probability": round(failure_probability, 4),
    })
```

Four things worth noticing:

1. `methods=["POST"]` — this route only accepts POST requests, since the caller is *sending data* (sensor readings), not just fetching a page.
2. `build_feature_row()` rebuilds the exact same 7-column layout the model was trained on — including the `Type_L` / `Type_M` dummy columns from `pd.get_dummies(...)` back in training. If a single column is out of order, the model will still run — and silently give you a wrong prediction. This is the single easiest mistake to make when deploying a model; matching column order is not optional.

3. `scaler.transform(row)`, not `scaler.fit_transform(row)` — we already fit the scaler during training. Fitting it again here, on a single live request, would compute a mean/std from one row, which is meaningless.
4. `jsonify(...)` — turns a Python dict into a proper JSON HTTP response, with the right `Content-Type` header. This is the "language" APIs speak.

**Try it right now**, with the server running locally (see Step 5 for how to start it):

```
curl -X POST http://127.0.0.1:5001/predict \
-H "Content-Type: application/json" \
-d '{
  "Air temperature [K]": 303.5,
  "Process temperature [K]": 312.0,
  "Rotational speed [rpm]": 1350,
  "Torque [Nm]": 65.0,
  "Tool wear [min]": 210,
  "machine_type": "L"
}'
```

```
{
  "failure_probability": 0.6924,
  "prediction": "Failure"
}
```

We also added a basic guard rail — if a field is missing, the API says so instead of crashing:

```
curl -X POST http://127.0.0.1:5001/predict -H "Content-Type: application/json" -d '{"ma
# -> HTTP 400: {"error": "Missing fields: ['Air temperature [K]', ...]"}
```

## Step 3 — Serving an HTML file from Flask

An API that only returns JSON is fine for another program, but a human wants a page to look at. Flask can serve HTML too — that's the `/` route:

```
@app.route("/")
def home():
    return render_template("index.html", raw_numeric_cols=RAW_NUMERIC_COLS)
```

`render_template()` looks for the file inside a folder literally named `templates/` — that's a Flask convention, not a setting we chose. Ours is `templates/index.html`.

The `raw_numeric_cols=RAW_NUMERIC_COLS` part passes a Python list into the template, which lets the HTML build its 5 input fields *from the same list the API uses* — so the form can never drift out of sync with what the model actually expects:

```
{% for col in raw_numeric_cols %}
  <label for="{{ col }}">{{ col }}</label>
  <input type="number" step="any" id="{{ col }}" required />
{% endfor %}
```

That `{% for %}` / `{{ }}` syntax is **Jinja2**, Flask's templating language — plain HTML, plus a small amount of Python-like logic for generating repeated or dynamic markup.

## Step 4 — Consuming the API from HTML

This is the piece that closes the loop: JavaScript, running in the browser, calling our own `/predict` route the same way `curl` did in Step 2.

From `templates/index.html`:

```
form.addEventListener('submit', async (e) => {
  e.preventDefault(); // stop the browser's normal page-reload sub

  const payload = { machine_type: document.getElementById('machine_type').value };
  numericCols.forEach(col => { payload[col] = document.getElementById(col).value; });

  const response = await fetch('/predict', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload)
  });
  const data = await response.json();

  resultBox.textContent = `${data.prediction} - failure probability: ${(data.failure_pr
  )});
```

`fetch()` is the browser's built-in tool for making exactly the kind of HTTP request `curl` makes from the terminal — this is the **entire mechanism** behind every "single page app" that updates without a full page reload. No framework, no library, no build step — just a POST request and a JSON response, read back into the page.

**Note the relative URL:** `fetch('/predict')`, not `fetch('http://127.0.0.1:5001/predict')`.

Because the HTML page and the API are served by the *same* Flask app, the browser already knows which host to talk to — this also means the exact same HTML keeps working unchanged once we move the app behind Nginx in Step 8.

## Step 5 — Running it locally

---

```
cd Flask-API-Deployment
python -m venv venv          # or reuse an existing env with these packages
source venv/bin/activate
pip install -r requirements.txt
python app.py
```

```
* Running on http://127.0.0.1:5001
```

**macOS note:** we run on port **5001**, not the more common 5000 — on modern macOS, port 5000 is already taken by the AirPlay Receiver (Control Center), so Flask can't bind to it. If you hit an "Address already in use" error on port 5000, this is almost always why.

Open <http://127.0.0.1:5001/> in a browser, fill in the form, click **Predict** — the page calls the API and shows the result, live, with zero page reload.

---

## Step 6 — Why the dev server isn't enough for production

---

Run `python app.py` and Flask itself warns you:

```
WARNING: This is a development server. Do not use it in a production deployment.
```

Three concrete reasons:

1. **It's single-threaded by default** — one slow request blocks every other user until it finishes.
2. **No process management** — if the app crashes (an unhandled exception, an out-of-memory error), nothing restarts it. Your API is just down.
3. **It's not designed to be exposed to the internet** — no protection against the kind of malformed/oversized requests a public server has to expect.

The fix is two separate, well-understood pieces of software, each doing one job: **Gunicorn** runs the app reliably, **Nginx** stands in front of it and faces the public internet.

---

## Step 7 — Gunicorn: a real WSGI server

---

Gunicorn ("Green Unicorn") is a **WSGI HTTP server** — its only job is to run your Flask app correctly, with multiple worker processes, and keep running.

Same app, same `app.py`, completely different way of starting it:

```
gunicorn -w 4 -b 127.0.0.1:8000 app:app
```

- `app:app` — "in the file `app.py`, use the object named `app`" (our `Flask(__name__)` instance). Nothing about our application code changes — Gunicorn just imports it instead of running its `if __name__ == "__main__"` block.
- `-w 4` — 4 worker processes. A request being slow in one worker no longer blocks the other three.
- `-b 127.0.0.1:8000` — listen only on localhost, port 8000. This is deliberate: Gunicorn is never exposed to the public internet directly — only Nginx will be allowed to reach it, on the same machine.

For it to survive server reboots and auto-restart on a crash, we don't run that command by hand — we hand it to **systemd** (Linux's service manager). See `deployment/predictive-api.service`:

```
ExecStart=/home/ubuntu/Flask-API-Deployment/venv/bin/gunicorn -w 4 -b 127.0.0.1:8000 ap
Restart=always
```

```
sudo cp deployment/predictive-api.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable predictive-api    # start automatically on boot
sudo systemctl start predictive-api
sudo systemctl status predictive-api    # confirm it's running
```

## Step 8 — Nginx: the public front door

Nginx is a **reverse proxy** — the public internet talks to Nginx (port 80/443), and Nginx quietly forwards each request to Gunicorn on `127.0.0.1:8000`, then relays the response back. The browser never sees Gunicorn at all.

Why bother with this extra hop instead of just exposing Gunicorn directly?

- Nginx is built to handle **thousands of concurrent slow client connections** efficiently (mobile users, slow networks) — Gunicorn workers are better spent running your Python code, not waiting on chatty clients.
- **SSL/HTTPS termination** happens here, in one place — the standard `certbot` tool wires up free HTTPS certificates directly into Nginx.
- **One machine can host many apps** behind one Nginx, each on its own internal port — the public only ever sees port 80/443.

`deployment/nginx_predictive_api.conf`:

```
server {
    listen 80;
    server_name your_domain_or_ip;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

The `proxy_set_header` lines matter more than they look: without them, Flask would think every single visitor's IP address is `127.0.0.1` (Nginx's own address), because that's who technically made the request to Gunicorn.

```
sudo cp deployment/nginx_predictive_api.conf /etc/nginx/sites-available/predictive-api
sudo ln -s /etc/nginx/sites-available/predictive-api /etc/nginx/sites-enabled/
sudo nginx -t          # validate the config before touching the live server
sudo systemctl reload nginx
```

## Step 9 — The full picture, and a testing checklist

Browser —HTTP:80→ Nginx —HTTP:8000 (localhost only)→ Gunicorn (4 workers) → Flask

- Flask/ `app.py` — **what** to do with a request (our code, unchanged since Step 2)
- Gunicorn — **how many** requests to handle at once, and keeping the app alive
- Nginx — **who's allowed to ask**, and the public-facing address

### Checklist, in order, when something isn't working:

1. `curl http://127.0.0.1:8000/` on the server itself — does Gunicorn respond? If not, the problem is Flask/Gunicorn, not Nginx.
2. `sudo systemctl status predictive-api` — is the service actually running? Check `journalctl -u predictive-api` for the Python traceback if not.
3. `sudo nginx -t` — is the Nginx config even valid?
4. `curl http://your_domain_or_ip/` from *outside* the server — if step 1 works but this doesn't, the problem is in Nginx or the server's firewall/security group, not your model.

## Summary

---

- **Serializing** ( `joblib.dump` / `joblib.load` ) is what separates *training a model* (slow, done once, in a notebook) from *using a model* (fast, done per request, in an API).
- **Flask** turns Python functions into URLs — one route ( `/predict` ) for the API, one route ( `/` ) for the HTML page, both defined the same way.
- **HTML consumes an API** with nothing more exotic than `fetch()` — a POST request out, a JSON response back, no page reload.
- **Gunicorn** is what makes the app reliable under real traffic; **Nginx** is what makes it safely reachable from the public internet. Neither replaces Flask — they wrap around it, in that order: `Nginx → Gunicorn → Flask` .
- None of this changed a single line of the model itself. That's the point — deployment is a separate skill from modeling, and it's the same skill regardless of whether the model behind `/predict` is today's AdaBoost or next month's something-else.